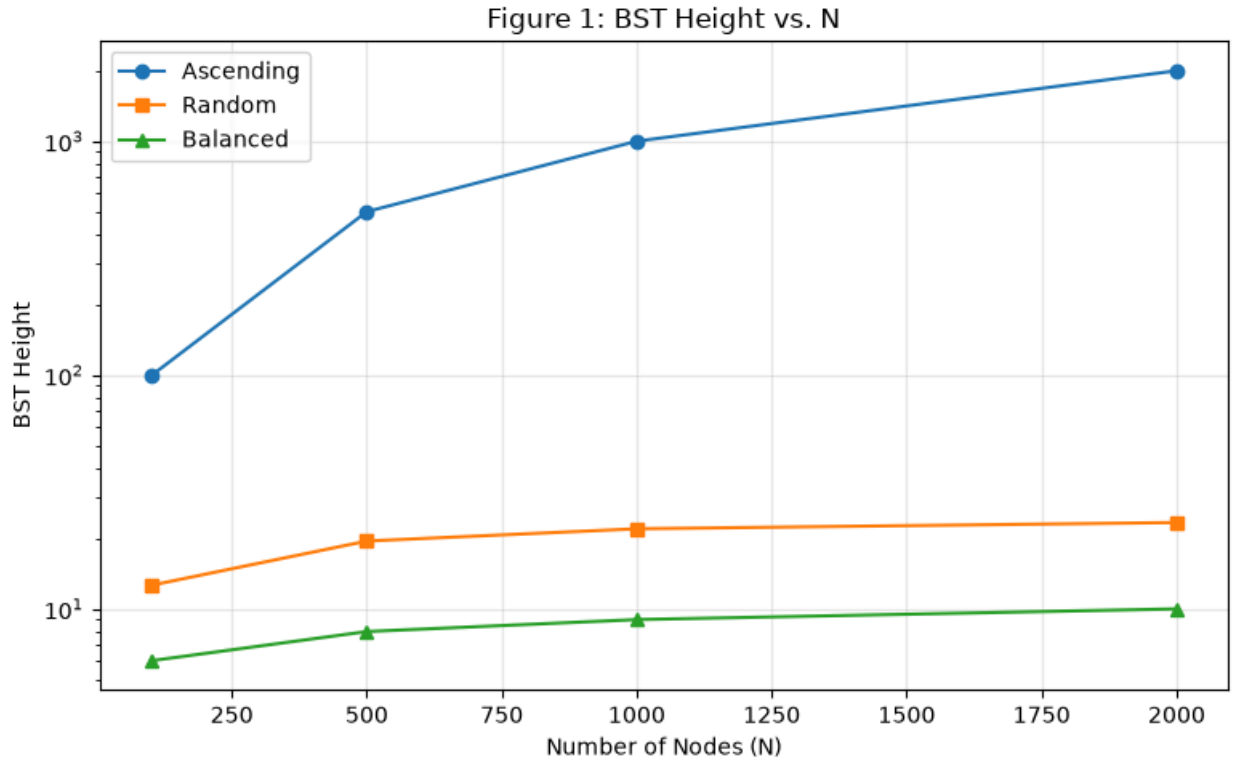
Part 2 Questions:

1. Which insertion strategy produces the tallest BST? Explain why.



Ascending produces the tallest BST, since each new insertion creates the new tallest height since each value is always greater than the previous.

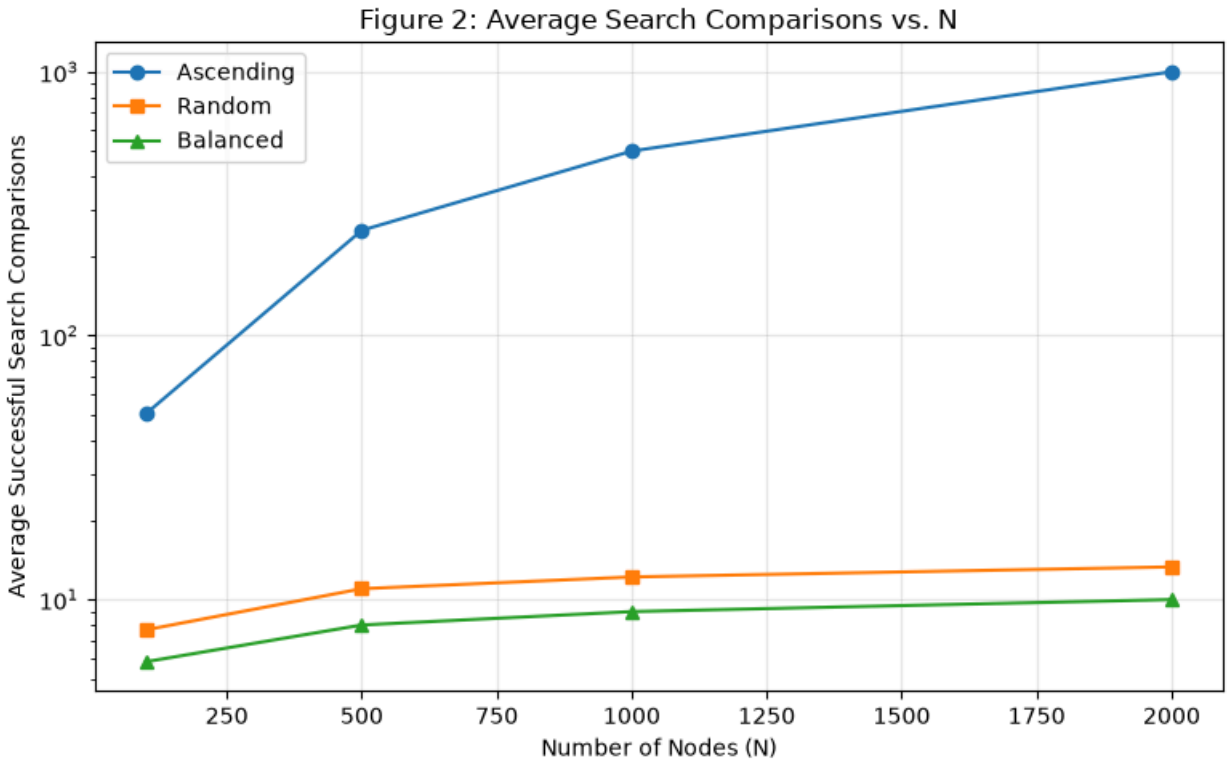
2. How does the height of the BST change as N increases for each strategy? Relate your observations to the theoretical time complexity of BST operations.

For ascending, the runtime is O(n), so as N increases the BST height increases by a factor of N, which we can see in the graph.

For random, we can see it is on the scale of O(log(n)), since random is the average case this makes sense.

For balanced, it is best case which is still on the scale of O(log(n)), but on the graph we can see it is less than the average case, even though they are on the same scale in terms of big O runtime.

3. Compare the average successful search comparisons across the three strategies. How closely do these results correspond to the tree heights?



They respond very closely with respect to the height of the tree. Even though height does not indicate search comparisons, it is still a good indicator since the worst case is still O(N), and every other case is on the scale of O(log(n)), and we can see that in the graph here. Balanced is still the lowest, then random, then ascending, where they are all still on the scale of O(N), and O(log(n)), but their values of their y-axis is less than that of their BST height.

4. Is a BST constructed using random insertion necessarily balanced? Explain based on your experiments.

No random insertion can make very un-balanced BSTS, and doesn't create any sort of balancing. From the graph we can see that random follows the same shape as balanced, but its number of search comparisons is greater, meaning that over time it starts getting closer to having a search runtime to O(log(n)), but the search runtime of a balanced tree will still have less comparisons.

Part 3 Questions: (Graph below answers)

1. How do the node-depth distributions differ among the three insertion strategies?

The ascending tree is a uniform distribution, because at every node depth, there is only 1 node, since each node after it is always greater, therefore creating a new max node depth when it gets inserted.

The random insertion creates a normal-like distribution, simply because of randomness. Around the mean of node-depth is the most number of nodes simply because numbers are more likely to be around there.

The balanced insertion follows an exponential distribution, which makes sense, since it creates a balanced tree, where the number of comparisons for a search of any node is of O(log(n)).

2. Why might two BSTs with similar heights have different average search performance?

Two BSTs with similar heights could have different average search performances due to one BST being created with an ascending list, while the other a random list with more entries. Both could have the exact same heights, yet their average search performances would vary greatly.

3. Explain the relationship between average node depth and average successful search comparisons. Can you express this relationship mathematically for a BST containing N distinct keys, assuming each key is searched exactly once?

The relationship between those two is that they are about equal in terms of O(log(n)), since on average, you would have to look through half of the tree if it is a leaf node, but the number of comparisons from that would be equal to the depth level + 1 to account for the root node being a depth level of 0:

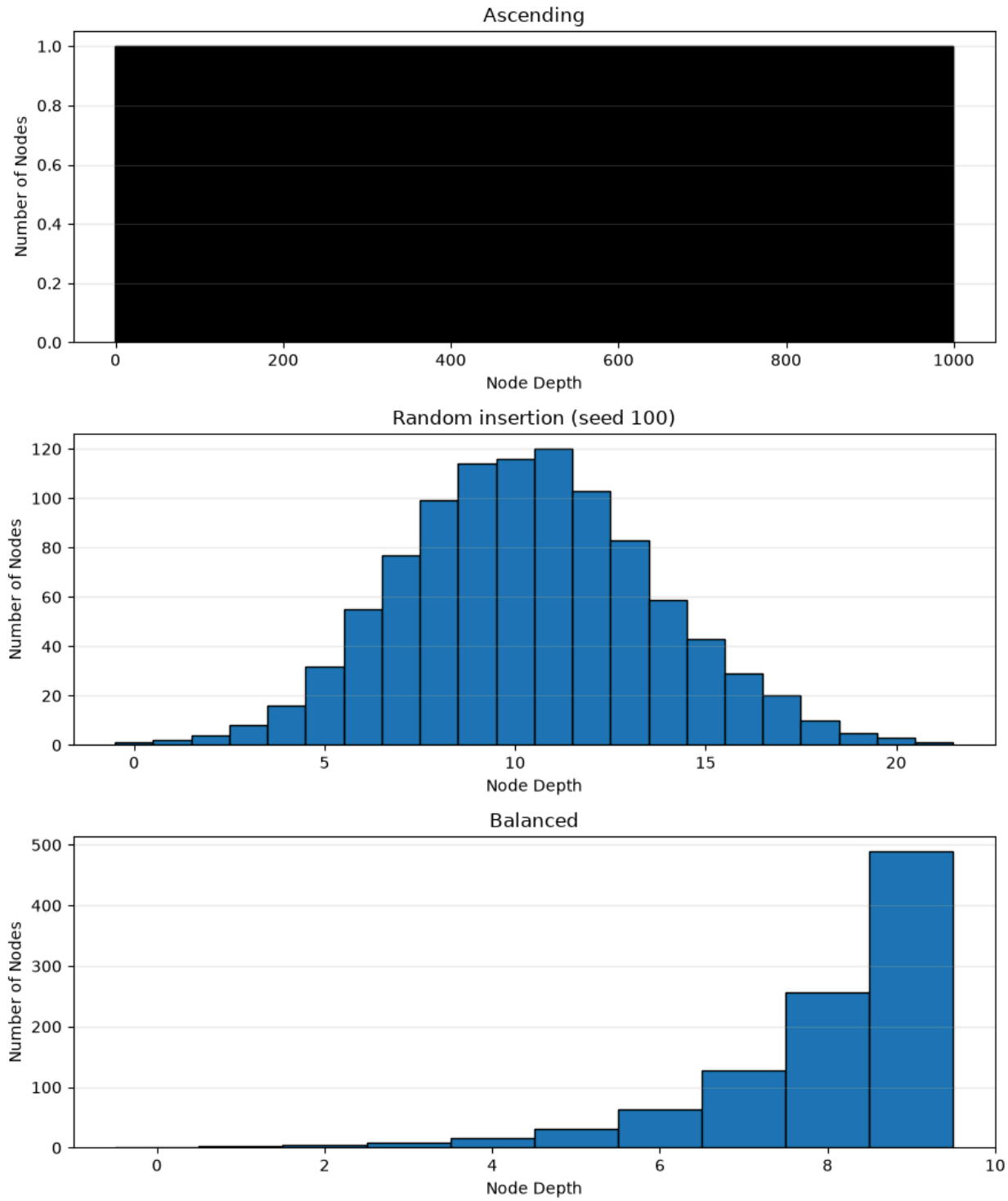
Let d_i = the depth of the key i.

The average node depth = 1/N * (sum(n, i = 1): d_i)

Comparison Average: 1/N * (sum(n, i = 1): (d_i) + 1)

Both of these are on the scale of O(log(n)).

Figure 3: Node-Depth Distribution (N = 1,000)

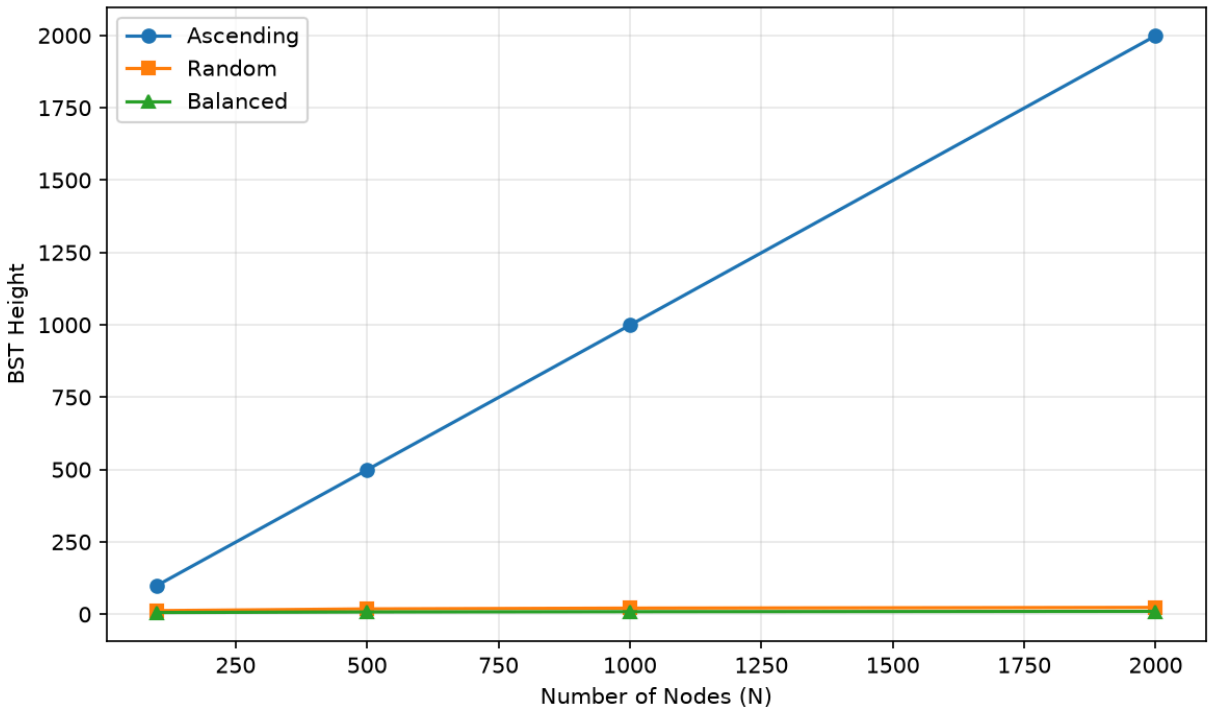


AI Usage:

I used AI to help me write code for the BST implementation, like showing me how self, current, node, and keys work coding wise. As well as showing me skeletons of how to graph and summarize the data. I also used AI to debug my errors for graphing and BST syntax where I had no clue what the error was, and the error was just misspelled or brackets instead of parenthesis. (I do not have the prompts as responses for those as I am not organized at all when it comes to my AI usage.)

At first the AI gave me skeleton code to graph the BST height and Average search comparisons in such a way that it looked like the graph below, which was not very helpful. and then I asked it again when I was reviewing my assignment and it recommended I could transform the y-axis into the log scale,which worked very well to visualize my findings.

Figure 1: BST Height vs. N



is there a way to show this graph better?

Option 2: Use a logarithmic y-axis

This keeps all three lines on the same graph while making smaller heights easier to see.

plt.figure(figsize=(10, 6))plt.plot(N_values, ascending_heights, marker='o', label="Ascending")plt.plot(N_values, random_heights, marker='s', label="Random")plt.plot(N_values, balanced_heights, marker='^', label="Balanced")plt.yscale("log")plt.xlabel("Number of Nodes (N)")plt.ylabel("BST Height (log scale)")plt.title("BST Height vs. Number of Nodes")plt.legend()plt.grid(True)plt.show()

Note: A logarithmic axis works best when all heights are positive.

Option 3: Improve the original graph with a zoomed inset

Keep the original graph but add a small zoomed-in section showing Random and Balanced. This is visually appealing, but requires a little more code.

What I would use for your assignment

- Option 1 if your goal is to clearly compare the performance of all three insertion orders.
- Option 2 if your assignment requires all three lines on one graph.

Your current graph already demonstrates an important result: ascending insertion produces a highly unbalanced BST, with height growing approximately linearly with N, while random and balanced insertion produce much shorter trees.